

RUDP Project: The Owner's Manual

Computer Networks project: Harshavardhan Mallela, Rohit Arya, Shloak Sinha. 9 Oct 2026.

We built a delivery service that keeps UDP's speed but adds TCP's reliability, taught it to adjust its own speed, tested it in a simulated four-router campus network and watched it work packet by packet in Wireshark. This manual explains each part the way a car manual explains the engine, the cruise control and the dashboard.

The car at a glance

Six parts, each with a car equivalent. The code lives in the GitHub repo `Capsturrrr/rudp`.

Part	What it does	Car equivalent	Where
RUDP	Sends data over UDP and makes sure every piece arrives, in order	The engine	<code>src/client.c</code> , <code>src/server.c</code> , <code>src/common.c</code>
Congestion control (AIMD)	Speeds up while the road is clear, slows down on loss	Automatic gearbox	inside <code>client.c</code>
Smart-RUDP	Learns the best speed with Q-learning instead of a fixed rule	Adaptive cruise control	<code>src/smart_client.c</code> , <code>src/rl_cc.h</code>
RUDP Chat	A two-way messenger on RUDP packets, with loss turned on	The test drive	<code>src/chat.c</code>
Packet Tracer network	Four routers, two VLANs, DHCP, an access list, failover	The road	<code>network/cfg/</code> , the <code>.pkt</code> file
Wireshark dissector	Decodes our custom packets so you can read them	The dashboard	<code>network/rudp.lua</code> , the <code>.pcapng</code> files

In one line: the engine moves data, the cruise control tunes its speed, the road is the network it travels over, and Wireshark is how we see what happened.

Start the engine

Run these in WSL, once, before any demo:

```
cd ~/projects/rudp
git pull
make all smart chat
```

RUDP with 20% packet loss. Open two terminals.

```
./bin/server      # terminal 1
./bin/client 20    # terminal 2
```

Look for "All 62 packets delivered" and the reassembled message on the server.

RUDP Chat. Start both before typing. The last number is the loss percentage.

```
./bin/chat 9001 9002 Alice 30 # terminal 1
./bin/chat 9002 9001 Bob 30 # terminal 2
```

Smart-RUDP vs AIMD. One run on the lossy path; compare the completion time and retransmissions.

```
gcc -DSERVER_PORT=9301 -o bin/server_9301 src/server.c src/common.c
./bin/server_9301 > /dev/null 2>&1 &
./bin/smart_client --mode aimd --port 9301 --delay 25 --jitter 10 --loss 2 --rate 800 --q
ueue 50 --scenario lossy --packets 1200 --qfile results/q_lossy.txt --seed 1001 | grep RE
SULT
./bin/smart_client --mode rl --port 9301 --delay 25 --jitter 10 --loss 2 --rate 800 --que
ue 50 --scenario lossy --packets 1200 --qfile results/q_lossy.txt --seed 1001 | grep RESU
LT
pkill server_9301
```

The full step-by-step demo script is in the file Demo_Steps.txt and in `docs/DEMO_RUNBOOK.md` in the repo.

The engine: RUDP

UDP is like dropping letters in a postbox: fast, but a letter can get lost, arrive twice or arrive out of order, and nobody tells you. RUDP adds the checks TCP has, written by us in C.

The packet. Every packet starts with a 13-byte header: sequence number (4 bytes, which packet this is), acknowledgment number (4, the next packet I expect), flags (1, what kind of packet), checksum (2, detects corruption) and payload length (2). Data follows. Flags: SYN starts a connection, ACK confirms, FIN ends it, DATA carries data.

The handshake. Client sends SYN, server answers SYN+ACK, client sends ACK. Only then does data flow. This is the same three-way handshake TCP uses.

Reliability. Each data packet has a number. The receiver accepts only the next expected number and replies with a cumulative ACK ("I have everything up to here"). Anything out of order is thrown away. The sender keeps a window of packets in flight (Go-Back-N). If no ACK arrives within 500 ms, it resends everything from the first unacknowledged packet. If three duplicate ACKs arrive, it resends right away (fast retransmit).

Congestion control. The window size decides how fast we send. It starts at 1 and doubles each round trip (slow start) until a threshold of 16, then grows by about 1 per round trip (congestion avoidance). On a loss it is cut: halved on three duplicate ACKs, reset to 1 on a timeout. This is AIMD, additive increase and multiplicative decrease.

Loss simulation. The client can drop packets on purpose (`./bin/client 20` drops 20%), so you can see the recovery working.

Cruise control that learns: Smart-RUDP

AIMD follows one fixed rule whatever the road looks like. Smart-RUDP replaces that rule with a small learning agent (tabular Q-learning) that picks how to change the window.

How it works. Every round trip the agent looks at three things: how much the delay has risen (RTT ratio), how much loss there was, and the current window size. That gives 80 situations (states). For each it chooses one of five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5. It is rewarded for throughput and punished for delay and loss, and it updates a table of scores as it goes (learning rate 0.15, discount 0.9).

What stays the same. Both modes share all the reliability code (Go-Back-N, adaptive timeout, fast retransmit). Only the window policy differs, so the comparison is fair.

How it was tested. Three emulated paths, each trained for 40 episodes, then 6 evaluation runs each: 5G-like (low delay), lossy wireless (2% loss) and satellite-like (150 ms delay). Loss, delay and a bottleneck queue are emulated inside the client.

The result, honestly. Smart-RUDP matched AIMD on the lossy path and was slower on the 5G-like and satellite-like paths, with somewhat fewer retransmissions. A simple learner does not beat a well-tuned rule. Say this plainly if asked; it is a real finding, not a failure.

Run it. See "Start the engine" above. Graphs are in [smart_graphs/](#), raw results in [results/](#).

The test drive: RUDP Chat

A two-way messenger built on the same packet format, so the protocol is something you can see working. Each message is one numbered packet. The receiver shows messages in order and sends ACKs; the sender resends anything lost.

Run it. Two terminals, ports swapped, last number = percent of packets the simulated network drops.

```
./bin/chat 9001 9002 Alice 30
./bin/chat 9002 9001 Bob 30
```

What to look at. Lines in brackets show the network losing a message and RUDP recovering: "network lost message #3", "timeout: resending", "3 duplicate ACKs: fast retransmit". Every message still arrives complete and in order. Ctrl+D quits and prints totals.

What it proves. At 30% loss in both directions, 15 and 12 messages arrived in order in our test.

Limits. It runs on one machine (localhost). No accounts or encryption. Loss is simulated inside the program.

The road: the Packet Tracer network

Four routers run OSPF (a protocol where routers share what they know and pick the shortest path). A student PC on VLAN 10 reaches the server across the core. In Packet Tracer the routers are labelled Router1, Router2, Router3 and Router0, configured as R1, R2, R3 and R4.

Link	Subnet	Notes
R1 Gi0/1 to R2 Gi0/0	10.0.12.0/30	R1 .1, R2 .2
R2 Gi0/1 to R3 Gi0/1	10.0.23.0/30	R2 .1, R3 .2
R1 Gi0/2 to R3 Gi0/2	10.0.13.0/30	primary path to the server
R2 Gi0/2 to R4 Gi0/0	10.0.24.0/30	R2 .1, R4 .2
VLAN 10 students	192.168.10.0/24	gateway R1 .1
VLAN 20 staff	192.168.20.0/24	gateway R1 .1
Server LAN	192.168.30.0/24	gateway R3 .1, server 192.168.30.10
Branch LAN	192.168.40.0/24	gateway R4 .1

What each feature does. VLANs split one switch into separate student and staff networks. Router-on-a-stick lets R1 route between them over one cable. DHCP hands out addresses automatically. The access list on R1 blocks students from reaching staff. OSPF picks routes: server LAN is metric 2 via R3, branch LAN is metric 3 via R2.

What we tested. R2 shows three FULL OSPF neighbours. PC0 traces to the server through 192.168.10.1, 10.0.13.2, then the server. The branch PC reaches the server. Pinging the staff PC from the student PC fails (access list), while the server still answers. When R1's link to R3 is shut down mid-ping, 70 of 71 pings still succeed and the route switches to R2 and R3.

Good to know. The cabling of R2 did not match the first plan. We found the real wiring with the command `show cdp neighbors` and changed R2's addresses to match.

Limit. Packet Tracer cannot run our C programs. The network design and the protocol are validated separately, and the protocol was tested on one machine with emulated paths.

The dashboard: Wireshark

Wireshark shows the actual packets on the wire. It does not know our custom protocol, so we wrote a small plug-in, `network/rudp.lua`, that decodes our 13-byte header into readable fields. (Wireshark already has a different built-in protocol called RUDP, so ours is named `smartrudp` internally; the packet list still says RUDP.)

Files. `rudp_capture_loss.pcapng` is a real capture of our client and server with packet loss, including the handshake. `smart_lossy_aimd.pcap` and `smart_lossy_rl.pcap` are traces exported by our own client for the AIMD and Smart-RUDP runs.

Open it (from WSL; Windows Wireshark cannot see the WSL network, so we open a saved file):

```
"/mnt/c/Program Files/Wireshark/Wireshark.exe" -X 'lua_script:C:\Users\Hanuman\Downloads\rudp_wireshark\rudp.lua' 'C:\Users\Hanuman\Downloads\rudp_wireshark\rudp_capture_loss.pcapng'
```

Two filters to show. Type in the bar at the top and press Enter.

- `frame.number <= 3` shows the handshake: SYN, SYN-ACK, ACK. Click packet 1 and expand RUDP to see sequence number, ack, flags, checksum and length.

- `smartrudp.seq == 1327198699` shows one data segment twice, at 0.0004 s and again at 0.501 s. That 500 ms gap is the timeout and Go-Back-N resend.

Other useful filters: `smartrudp.flags.data == 1` (data only) and `smartrudp.flags.ack == 1` (ACKs only; repeated ack numbers are duplicate ACKs).

Warning lights

Symptom	Likely cause	Fix
<code>make</code> says smart or chat target not found	Old copy of the repo	<code>cd ~/projects/rudp && git pull</code>
Server prints nothing when output goes to a file	Output is buffered	Run it in a terminal, or use <code>stdbuf -oL ./bin/server</code>
Chat shows no messages	The other terminal was not started, or the ports are not swapped	Start both; first port is yours, second is the peer's
Smart-RUDP run hangs or errors on bind	A leftover server still holds the port	<code>pkill server_9301</code> and run again
Wireshark error "two protocols with the same name"	Old <code>rudp.lua</code> loaded twice	Delete any <code>rudp.lua</code> in <code>%APPDATA%\Wireshark\plugins</code> and use the command above
Packets show as plain UDP in Wireshark	Dissector not loaded	Analyze, Decode As, UDP port 8888 = RUDP
Packet Tracer interface up but protocol down	The other end is not configured yet	Configure it, wait 30 to 40 seconds of simulated time
<code>show ip ospf neighbor</code> is empty	OSPF has not converged, or an address does not match the cable	Fast Forward Time; show <code>cdp neighbors</code> to check the real wiring
Ping to the server is slow after the failover test	R1 Gi0/2 is still shut down	<code>interface gigabitEthernet0/2</code> , then <code>no shutdown</code>
First ping of a run times out	ARP is resolving	Normal; run it again

Questions you will be asked

How is this different from a normal UDP lab program? A lab program just sends datagrams. Ours adds sequence numbers, checksums, acknowledgments, retransmission, a sliding window, a handshake and congestion control, which is what TCP gives, built by us on top of UDP.

Why not just use TCP? TCP is inside the operating system and cannot be changed. Writing our own makes every mechanism visible and tunable. QUIC and HTTP/3 do the same in practice.

Did Smart-RUDP beat AIMD? No. It matched AIMD on the lossy path and was slower on two paths, with somewhat fewer retransmissions. A simple learner does not beat a tuned rule; the honest result is in the report.

Does the protocol run inside Packet Tracer? No. Packet Tracer cannot run C programs. The network design is validated in Packet Tracer; the protocol is validated on one machine with emulated loss and delay. The report says this.

Is the loss real? In the experiments and the chat it is simulated inside the program. We also captured a real loopback run with tshark and loss applied, which is the Wireshark capture.

What is the purpose? A readable, tunable reliable transport for campus users sending data to a server across a routed network, tested against routing, access policy and link failover.

Why Go-Back-N and not selective acknowledgments? Go-Back-N is simpler to build and explain. It wastes bandwidth under loss, which is listed as a limitation and as future work.

What happens if a router link fails? OSPF reroutes. In our test, R1's link to R3 was shut down during a ping: 70 of 71 pings succeeded, and the route moved to the longer path through R2 and R3.

Glossary

Term	Plain meaning
UDP	Sends packets with no guarantee they arrive
TCP	Reliable delivery built into the operating system
ACK	A receipt saying a packet arrived
Sequence number	The number of a packet, so order can be checked
Checksum	A small number that detects a damaged packet
Handshake	Three packets (SYN, SYN-ACK, ACK) that open a connection
Window	How many packets may be in flight without an ACK
Go-Back-N	On a loss, resend everything from the first lost packet
Fast retransmit	Resend at once after three duplicate ACKs, without waiting for a timeout
RTT	Round-trip time: how long a packet and its ACK take
Congestion control	Slowing down when the network is overloaded
AIMD	Grow the window slowly, cut it sharply on loss
Slow start	Doubling the window each round trip at the beginning
Q-learning	A learner that scores each action in each situation and improves with rewards
OSPF	A protocol where routers share links and pick the shortest path
VLAN	A separate virtual network on one switch
DHCP	Hands out IP addresses automatically
ACL	Access list: a router rule that permits or blocks traffic
Packet Tracer	Cisco's network simulator

Term	Plain meaning
Wireshark	A tool that shows packets in detail
Dissector	A plug-in that teaches Wireshark to read a protocol
pcap	A file of captured packets
netem	Linux tool that adds delay and loss to a network interface